

Question 5.

Write a program using suitable cryptographic library functions to implement digital signatures using both Elliptic Curve Digital Signature Algorithm (ECDSA) and RSA Digital Signature.

- a) Generate an ECDSA public–private key pair using a standard elliptic curve such as SECP256R1.
- b) Generate an RSA public–private key pair with a key size of at least 2048 bits.
- c) Accept a message from the user.
- d) Generate a digital signature for the message using:
 - a. ECDSA with SHA-256
 - b. RSA with SHA-256 and an appropriate signature padding scheme such as RSA-PSS
- e) Display the generated signatures.
- f) Verify both signatures using the corresponding public keys.
- g) Modify the original message and demonstrate that signature verification fails for the modified message. Use suitable library functions available in Python's cryptography library or an equivalent cryptographic library.

```
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import ec, rsa, padding
from cryptography.hazmat.primitives.serialization import (
    Encoding, PrivateFormat, NoEncryption
)

message = input("Enter message: ").encode()

# ECDSA Key Generation
ecdsa_private_key = ec.generate_private_key(ec.SECP256R1())
ecdsa_public_key = ecdsa_private_key.public_key()

# RSA Key Generation
rsa_private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
rsa_public_key = rsa_private_key.public_key()

# ECDSA Signature
ecdsa_signature = ecdsa_private_key.sign(
    message,
    ec.ECDSA(hashes.SHA256())
)
```

```

)

# RSA Signature using RSA-PSS
rsa_signature = rsa_private_key.sign(
    message,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

print("\nECDSA Signature:")
print(ecdsa_signature.hex())

print("\nRSA Signature:")
print(rsa_signature.hex())

# Verify ECDSA signature
try:
    ecdsa_public_key.verify(
        ecdsa_signature,
        message,
        ec.ECDSA(hashes.SHA256())
    )
    print("\nECDSA Verification: Successful")
except Exception:
    print("\nECDSA Verification: Failed")

# Verify RSA signature
try:
    rsa_public_key.verify(
        rsa_signature,
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
    print("RSA Verification: Successful")
except Exception:

```

```

    print("RSA Verification: Failed")

# Modified message
modified_message = input("\nEnter modified message: ").encode()

# Verify ECDSA with modified message
try:
    ecdsa_public_key.verify(
        ecdsa_signature,
        modified_message,
        ec.ECDSA(hashes.SHA256())
    )
    print("Modified ECDSA Verification: Successful")
except Exception:
    print("Modified ECDSA Verification: Failed")

# Verify RSA with modified message
try:
    rsa_public_key.verify(
        rsa_signature,
        modified_message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
    print("Modified RSA Verification: Successful")
except Exception:
    print("Modified RSA Verification: Failed")

```

```

● yashsonawane@yashs-MacBook-Air cryptography % python3 que1.py
○ yashsonawane@yashs-MacBook-Air cryptography % python3 que1.py
Enter message: hello Crypto

ECDSA Signature:
3045022100ea1921b188bd0d528aedd6f7410e9952b9a7556d36ebacf51fd05b8661f7586c022078dac1d6481742d3986e79c74a610ad17b560d80426b2dd6cd5fd186121e6139

RSA Signature:
8cc4c6ac98932243a11fcf463133021ceeb3a256ed004316b3b5dbc012724dec30970c1b421b328832d7ca5d0ef662a8c79c338af0c178df65981a5a72679848c2692a03fe8f6224e82b64446f97
45ebd9a18c7f71c02440dfbcdb21794c09435c3de5b1e0688c46adeec0cda00f63c5556d6d883787185ee11f3742586499f7813d68ca9bae844eb25173ab491f35d1b4e83591d03e0b28ef407ba
ac495e31db8b9b287e4cfb5b1cf5cd6712abdcdaa37f3935dc48350212faa270267faa33015262459c36ab3ae0ecae0e57c7bad0ba7983a7cc43bbe2e08e468eac14bf6b9941cfc7563822a635acf
19b7bac64d7d1434e05f05ebaa5b08b2f89e882df9a3

ECDSA Verification: Successful
RSA Verification: Successful

Enter modified message: 

```